LLM エージェントにおける履歴要約方式と終了制御の総トークン数評価

著者未設定

Evaluating Total Token Usage of History Summarization Methods and Termination Control for LLM Agents

Author details pending

所属・会員種別・連絡先：未設定

あらまし　LLM エージェントの履歴要約方式と API 実行記録に基づく終了制御を独立に評価する．要約生成を含む全モデル呼出しを計測し，既知課題の成功を保つ総トークン数の削減を確認した．呼出し配置と完了後の生成に着目した実行管理の比較結果を示す．

キーワード　LLM エージェント，ハーネス，履歴要約，終了制御，トークン使用量

1. はじめに

日常業務のデータ整理や事務処理の負担を軽減するため，大規模言語モデルによる作業支援が研究されている．以下では大規模言語モデルを LLM と呼ぶ．文章の回答生成に加え，外部アプリケーションを操作する複数手順の実行も対象となる．例えば写真の整理や精算では，対象の検索，状態の変更，結果の確認を連続して行う必要がある．推論と行動を繰り返す ReAct[1]や，コードを行動として実行する CodeAct[2]は，このような対話的処理を支える技術である．

一方，実行が長くなると，過去の操作と観測を次の生成へ繰り返し入力する費用が増える．また，作業が終わった後も確認や生成を続ければ，成果を増やさずに資源を消費し得る．履歴を短くするだけでは必要な情報を失うおそれがあり，早く止めるだけでは未完了となり得るため，最終成功と実行全体の費用を同時に調べる必要がある．

本稿では，モデルを再学習せずに実行を管理するハーネスについて，履歴要約の呼出し配置と終了制御を評価する．履歴要約では，要約を生成する時点，行動生成と呼出しを分けるかどうか，要約機会の上限を比較軸とする．終了制御では，実行中に取得できる API の要求と応答を記録し，この API 実行記録に基づく制御を通常継続と比較する．両者を独立した実験として実行し，要約生成を含む全モデル呼出しの総トークン数と，課題の最終成功を調べる．

2. 関連研究

入力圧縮では，長文中の情報の位置が性能に影響すること[3]を踏まえ，LLMLingua[4]は予算制御とトークン選別，LLMLingua-2[5]は蒸留データによるトークン分類，LongLLMLingua[6]は長文の重要情報を残す圧縮を扱う．

長期実行では，ACON[7]は観測・履歴の圧縮方針，TokenPilot[8]は入力圧縮とキャッシュ継続性を扱う．Generative Agents[9]は経験の記憶・抽象化・検索，Reflexion[10]は言語的フィードバックを後続試行に利用する．本稿は，要約を専用の呼出しで生成する方式と，行動と同じ応答に含める方式を比べ，要約生成も含む課題全体のトークン使用量を評価する．

終了制御では，Runaway[11]が終了指示と外部の完了検証を検討している．本稿では，既知の要求形式について API 実行記録から完了条件を検査する．通常継続との比較では，最終成功，API 要求数，総トークン数を評価する．

3. 提案手法

ハーネスは，モデルの外側で入力と実行の継続を管理するプログラムである．本稿では，既存の要約を使って入力履歴を更新する処理と，API 実行記録を使って次の生成の要否を判断する処理を構成する．図 1 の実験 A には要約と行動生成の呼出し配置を，実験 B には完了判定に基づく終了制御を示す．両者は別々に評価し，組合せによる削減は扱わない．

比較する三方式には二つの対応関係がある．専用 1 回方式と専用反復方式では，要約と行動生成の呼出しを分ける構成を共通とし，要約機会の上限を変える．専用反復方式と行動併記方式では，機会の上限を揃え，候補を別の呼出しで生成する構成と，行動と同じ応答で生成する構成を比べる．候補の採否は，いずれもモデルの外側で決める．このように，生成を担うモデル

と入力の更新を担うハーネスの分担を保ちながら，候補生成の配置と機会の上限を変えた構成を評価する．

3.1　入力履歴を短くする制御

まず，課題とこれまでの履歴をモデルへ渡し，次に実行する行動を生成する．ハーネスは生成されたプログラムを実行し，その行動と結果を履歴に追加する．行動生成から実行結果の記録までを 1 ターンとする．

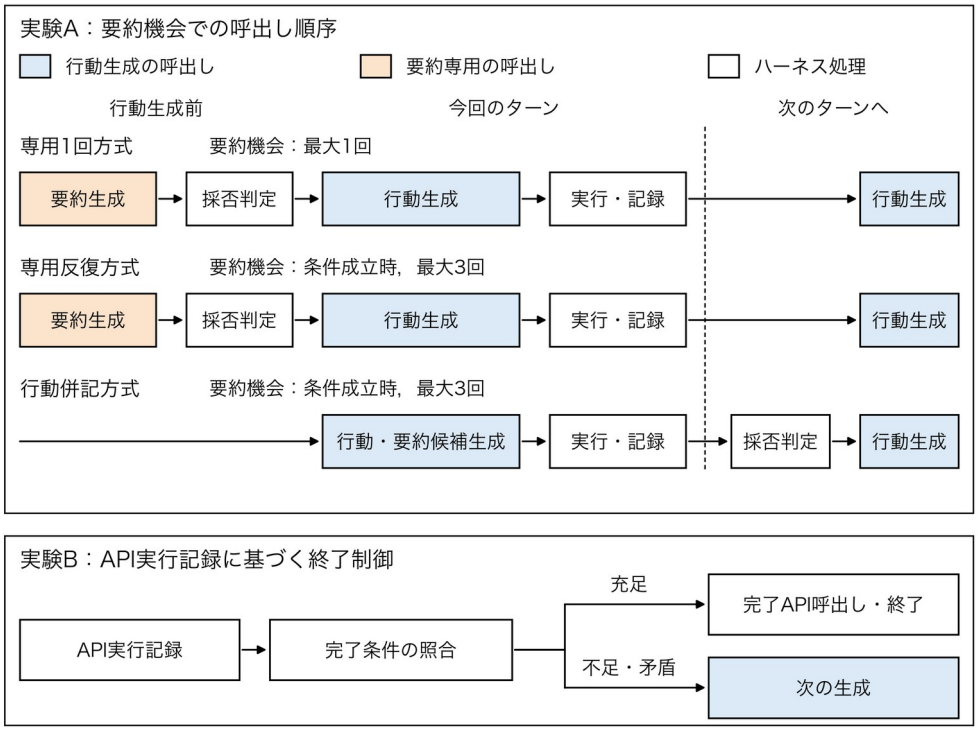


図1　履歴要約方式と終了制御

この繰返しで長くなる入力を抑えるため，保存した全履歴は残したまま，モデルへ渡す古い履歴を要約に置き換える．生成した要約を，検査して採用する前は要約候補と呼ぶ．本稿では，要約を生成するためだけに行うモデル呼出しを専用呼出しとし，次の 3 方式に比較用の名称を付ける．

専用 1 回方式は，要約専用のモデル呼出しを最大 1 回追加する方式である．入力長が閾値に達し，要約できる古い履歴がある最初の要約機会に候補を生成する．候補を検査し，採用する場合は要約と未要約の履歴を次の行動生成へ渡す．要約機会は最大 1 回とし，その機会の終了後は新たな要約機会を設けない．

専用反復方式は，要約専用の呼出しを必要に応じて繰り返す方式である．最初の要約機会では，専用 1 回方式と同じ処理を行う．以後，入力長と新たに完了したターン数の条件を満たすと，その時点で未要約の古い履歴を要約対象とする．採用済みの要約がある場合は，その要約も対象に含める．要約機会は最大 3 回とし，各機会で候補の採否を決めてから次の行動を生成する．

行動併記方式は，行動を生成するモデルの同じ応答に，行動と要約候補を出力する方式である．要約専用の呼出しは追加せず，最大 3 回の要約機会を設ける．この応答内の全プログラムを実行し，結果を記録してから，次の行動生成へ進む前に候補の採否を決める．候補には，その応答で生成した行動の実行結果は含めない．その行動と実際の結果は原文のまま次の入力に残す．

どの方式でも，要約には完了した作業，次の操作に使う情報，残作業，不確かな点の四つの欄を用いる．候補が得られたら，ハーネスが形式と文字数を検査する．さらに，元の指示と課題，要約，未要約の履歴を含む次回入力全体のトークン数を比べ，候補を採用しない場合より厳密に少なくなることを確認して採用する．要約候補を生成する時点で，直近 2 ターンを原文のまま残す．検査するのは構造と入力長であり，要約内容の正しさを保証するものではない．

完了した作業の欄には実行済みの内容を，次の操作に使う情報の欄には識別子，API の呼出し方，保持された変数などを残すよう指示する．残作業と不確かな点の欄では，未実施の操作と未確認の事項を区別する．API の正常応答だけから課題の完了を推定しないよう求める．

元の指示と課題は要求の基準として残し，直近の行動と API 応答も原文で渡す．これにより，古い履歴の提示方法を変えながら，要求と直近の実行結果を参照できる構成とする．

採用条件を満たさなければ，それまでの要約と未要約の履歴を維持する．まだ要約を採用していない場合は，要約前の履歴を使い続ける．要約機会は候補の生成を試みる機会であり，採用回数とは区別する．不採用の場合も 1 回の機会として数え，その場で候補の再生成は行わない．専用 1 回方式は機会の上限を定める名称であり，生成や採用が 0 回となる場合もある．入力長とターン数の閾値は 4.1 に示す．保存した全履歴は，入力を短くした後も別に残す．

専用呼出しでは候補の生成に 1 回のモデル呼出しを使うが，行動併記方式では行動生成の呼出しを共有する．一方，採用回数は入力を実際に更新した回数である．本稿では要約機会数，採用回数，モデル呼出し数を明確に区別して扱う．採用後も新しい行動と結果は未要約の履歴として追加され，次の要約機会まで入力に残る．

3.2　作業完了後の生成を省く制御

終了制御は，API 実行記録から課題の完了を検査し，根拠が揃った時点で次の生成を省く処理である．この検査の段階を完了判定と呼ぶ．写真を作成時期に応じた指定先へ移す課題を対象とし，既知の一つの要求形式に対応する検査規則を手書きで用意する．まず，モデル応答内の全プログラムを実行し，API の要求と応答を記録する．次に，移動前の写真一覧と各写真の作成日時，API から取得した現在年を確認する．これらに基づいて，全ての対象が指定先へ移され，元の名前が保たれているか照合する．完了の根拠が初めて揃った時点で完了 API を 1 回呼び出し，次の生成を省く．

完了判定は，モデルの完了宣言ではなく，API 実行記録に対象全件の処理を確認できる情報があるかを調べる．通常継続は，この判定による終了を行わず，通常の実行手順を続ける比較条件である．

対象集合を移動前に確定するのは，一部の移動だけで完了と判断することを避けるためである．作成日時と API から得た現在年は，写真ごとに指定された移動先を照合する際に使う．移動後の場所に加えて元の名前の保持を調べることで，指定先へ移すという要求と，名前を変えないという要求を別々に確認する．これらの根拠はモデルの推測や計算機の時刻で補わず，API 実行記録から取得する．

対応する要求形式から外れる場合や，判定に必要な記録が不足・矛盾する場合は，通常どおり次の生成へ進む．判定には，その時点で得られた API 実行記録だけを使い，公式評価器の正解は使わない．履歴要約との組合せ効果は評価しない．

3.3　総トークン数と課題の成功

1 回の入力を短くしても，要約生成やその後の行動生成に使用するトークンが増えれば，課題全体の使用量は増え得る．そこで，要約後の入力だけを比べず，課題開始から終了までを全方式に共通する集計範囲とする．

条件 p で実行したモデル呼出しの集合を $\mathcal{C}_p$ とする．各呼出し c の入力トークン数を I_c，出力トークン数を O_c とし，実行全体の総トークン数 T_p を式(1)で求める．

$$T_p = \sum_{c \in C_p} \left(I_c + O_c \right) \tag{1}$$

要約にも生成費用がかかるため，行動生成と要約生成の両方を集計する．不採用となった要約の生成や失敗時の呼出しも，取得できた使用量を計上する．使用量が不明な場合は 0 とせず，既知分の小計と不明件数を記録する．

課題の成功は，実行終了後の状態に対する AppWorld[12]の公式評価器で判定する．評価器の正解や詳細は実行中のモデルへ返さない．方式間では成功数と，各課題の成功の維持を確認する．

同じ課題を処理した方式同士で成功と総トークン数を対応させる．課題をまたぐ合計では減少と増加が相殺されるため，実験 A では課題別の値も示す．

本指標はキャッシュされた入力も含む論理トークン数であり，金銭課金や実行時間を直接表すものではない[8]．要約生成を含む費用評価は ACON[7]にも用いられている．実験 A と B は独立に集計する．履歴要約は次回入力の長さ，終了制御は次の生成の必要性を管理する．判断に使う材料と変更する対象が異なるため，それぞれの制御と対照を独立に比較する．

4.　実験 A：要約の呼出し配置

4.1　条件

写真操作，精算，返金，CSV 処理の既知 4 課題に対し，専用 1 回方式，専用反復方式，行動併記方式の各方式を各課題で 1 回ずつ実行した．4 課題に 3 方式を適用し，計 12 実行とした．モデルは全条件で MiniMax-M3 を用い，GGUF 形式の IQ4_XS 量子化モデルを llama.cpp b11669 で実行した．文脈長は 65,536 トークンである．乱数の初期値を表す seed は，モデ

ル生成の基点を 57，環境を 42 とした．生成側は呼出しの役割と順序に従い seed を増分した．

要約機会を設ける入力長の閾値は 16,384 トークンとし，要約時には直近 2 ターンを原文のまま保持する．専用反復方式と行動併記方式では，前回の要約機会が終了してから新たに 3 ターン以上が完了することを要求する．採用と不採用のどちらでも，その機会の終了時点を次回の起点とする．初回の起点は課題開始である．要約は合計 8,192 文字以下とし，次の行動生成へ渡す入力全体が短くなる場合に採用する．要約専用の呼出しでは出力上限を 4,096 トークンとし，その使用量も式(1)へ含める．

各実行の上限はモデル呼出し 80 回，総トークン数 450,000，作業時間 1,800 秒とし，環境 API は写真 160，他 3 課題 80 要求とした．生成の温度は行動生成で 1.0，要約専用の生成で 0.7 とした．並列実行数は 1，KV キャッシュは q8_0，計算機は Apple M3 Ultra，メモリは 256 GiB である．実装と記録を公開する．[1]

4.2 全 4 課題の結果

表 1 に 4 課題の結果を示す．全 12 実行で課題に成功し，使用量の欠測はなかった．

表 1　実験 A の課題別総トークン数

課題	専用 1 回方式	専用反復方式	行動併記方式
写真操作	139,871	139,871	129,374
精算	211,110	211,110	211,110
返金	102,087	102,087	102,087
CSV 処理	440,461	332,588	394,351
合計	893,529	785,656	836,922

要約生成を含む 4 課題の合計は，専用反復方式が 785,656 トークンで 3 方式中最少だった．専用 1 回方式の 893,529 トークンに対し 107,873 トークン少なく，削減率は 12.07%だった．この差は CSV 処理だけで生じ，残る 3 課題では両方式の値が一致した．

写真操作では，専用 1 回方式と専用反復方式は 139,871 トークンで，行動併記方式は 129,374 トークンだった．行動併記方式は他の 2 方式より 10,497 トークン少なかった．要約の採用回数はいずれも 1 回だった．同じ 1 回の要約採用でも，課題全体の総トークン数には差があった．

精算の総トークン数は全方式で 211,110 トークンだった．いずれも要約の採用回数は 0 回で，方式による総量の差はなかった．

返金では，全方式で 102,087 トークンを使用し，要約の採用回数は 0 回だった．精算と同様に，成功と総トークン数は全方式で一致した．

CSV 処理では，専用 1 回方式は 440,461，専用反復方式は 332,588，行動併記方式は 394,351 トークンだった．専用反復方式は専用 1 回方式より 107,873 トークン少なく，4 課題合計の削減はこの課題で生じた．要約の採用回数は専用 1 回方式が 1 回，専用反復方式と行動併記方式が各 2 回だった．専用反復方式と行動併記方式の採用回数は等しかったが，総トークン数は一致しなかった．

行動併記方式は 4 課題合計で専用 1 回方式より 6.34%少なかったが，専用反復方式より 51,266 トークン多く，増加率は 6.53%だった．専用反復方式との比較では，写真操作で 10,497 トークン減る一方，CSV 処理で 61,763 トークン増えた．要約専用の呼出しを省く構成でも，課題全体の総トークン数が最少になるとは限らなかった．表 1 の課題別の値と合計を併記することで，異なる課題で生じた減少と増加を分けて確認できる．

5. 実験 B：API 実行記録に基づく終了制御

5.1 条件

同じ写真課題について，API 文書の取得を必須とする文書必須条件と，公開 API 名からのアクセスを許す名前参照条件を設けた．各条件で，通常継続と，API 実行記録に基づく終了制御を比較した．終了制御は，事前に実行・保存した通常継続と同じ初期状態，モデル，seed，モデルへの指示とツール説明，予算を用いて各 1 回実行した．

seed の基点は 42 であり，実験 A とは別の履歴・設定である．同じ MiniMax-M3 IQ4_XS と llama.cpp b11669 を用い，各実行の予算はモデル呼出し 80 回，450,000 トークン，1,800 秒，API160 要求とした．

完了判定と完了通知は 3.2 に従う．判定，記録保存，完了通知を含む実行時間を記録し，完了通知を API 要求数に含めた．

行動の生成は LLM が担い，記録の検査はハーネスの規則で行う．この検査のために別のモデル呼出しは設けない．完了の根拠が揃えば次の行動生成を省き，根拠が足りなければ同じ実行を継続する．

1 https://github.com/sophiria1225/AI-

終了以前の生成過程の違いが比較に含まれていないか確認するため，終了制御側で実行したモデル呼出しの要求と応答が，通常継続側の先頭から同じ範囲と完全に一致することを事前の照合基準とした．API 操作の記録と最終状態も比較した．

5.2　成功を保った総トークン数の削減

表 2 に結果を示す．両 API 条件で課題の成功を保ち，終了制御により総トークン数を削減した．文書必須では 130,439 から 91,116 へ 30.15%，名前参照では 151,664 から 112,233 へ 26.00%減った．全 4 実行で公式評価器による成功を確認し，使用量の欠測はなかった．各条件でモデル呼出し 2 回と API 要求 4 回を削減した．完了通知を含めて処理を終え，移動後の 4 回の確認 API と，それに伴う生成を省いた．

表 2　実験 B の総トークン数と呼出し回数

API 条件	終了方法	総トークン数	モデル呼出し数	API 要求数
文書必須	通常継続	130,439	15	104
文書必須	終了制御	91,116	13	100
名前参照	通常継続	151,664	14	103
名前参照	終了制御	112,233	12	99

実行した API 操作などの前半の記録と，実行終了時の 13 個のデータベースのハッシュ値は対照と一致し，最終状態を保った削減を確認した．一方，要求・モデル応答全文の完全一致基準は満たさなかった．このため，観測した差の全てを終了制御のみの効果とは断定せず，既知の要求形式に対する各 1 回の比較として扱う．平均的な因果効果や汎用的な終了保証を示す評価ではない．

6. むすび

既知 4 課題全てで成功を保ち，専用反復方式の総トークン数は専用 1 回方式を 12.07%下回った．この差は CSV 処理によるものであり，他 3 課題の値は一致する．行動併記方式の合計は専用反復方式より 6.53%多い．独立に評価した写真課題の 2 つの API 条件では，終了制御を用いた実行で成功を保ち，通常継続に比べて 26.00–30.15%の削減を確認した．本評価は，要約生成を含む全モデル呼出しの計測と，API 実行記録から得られる完了の根拠に基づく実行管理の意義を示す．

本稿は，既知課題と単一モデル・量子化設定における少数の seed・試行による開発実験を事後に整理した評価である．未見課題の品質維持や統計的非劣性は評価していない．

文　献

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” Proc. ICLR, May 2023.

[2] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” Proc. ICML, PMLR, vol.235, pp.50208–50232, July 2024.

[3] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” Trans. ACL, vol.12, pp.157–173, 2024. doi: 10.1162/tacl_a_00638.

[4] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing prompts for accelerated inference of large language models,” Proc. EMNLP, pp.13358–13376, Dec. 2023. doi: 10.18653/v1/2023.emnlp-main.825.

[5] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang, “LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression,” Findings of ACL: ACL 2024, pp.963–981, Aug. 2024. doi: 10.18653/v1/2024.findings-acl.57.

[6] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, “LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression,” Proc. ACL, vol.1, pp.1658–1677, Aug. 2024. doi: 10.18653/v1/2024.acl-long.91.

[7] M. Kang, W.-N. Chen, D. Han, H. A. Inan, L. Wutschitz, Y. Chen, R. Sim, and S. Rajmohan, “ACON: Optimizing context compression for long-horizon LLM agents,” Proc. ICML, PMLR, vol.306, pp.55321–55375, July 2026.

[8] B. Xu, Z. Xue, D. Chen, C. Fu, C. Wu, C. Huang, C. Jiang, J. Fang, X. Deng, Y. Chen, Y. Yao, X. Wang, J. Shang, H. Chen, and N. Zhang, “TokenPilot: Cache-efficient context management for LLM agents,” Findings of ACL: EMNLP 2026, pp.19034–19049, Oct. 2026.

[9] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” Proc. ACM UIST, article 2, pp.1–22, Oct. 2023. doi: 10.1145/3586183.3606763.

[10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” Advances in Neural Information Processing Systems, vol.36, pp.8634–8652, 2023. doi: 10.52202/075280-0377.

[11] Q. Lu, L. Ding, S. Cao, X. Liu, K. Zhang, J. Zhang, and D. Tao, “Runaway is ashamed, but helpful: On the early-exit behavior of large language model-based agents in embodied environments,” Findings of ACL: EMNLP 2025, pp.24014–24027, Nov. 2025. doi: 10.18653/v1/2025.findings-emnlp.1304.

[12] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian, “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” Proc. ACL, vol.1, pp.16022–16076, Aug. 2024. doi: 10.18653/v1/2024.acl-long.850.

Abstract
We independently evaluated history summarization methods and termination control based on API execution records for LLM agents. Counting all model calls, including dedicated summarization, showed reductions in total tokens while preserving success on known tasks under the evaluated conditions.
Key words: LLM agents, harness, history summarization, termination control, token usage